

SYNOPSIS

Title

FoodBridge – AI-Assisted Surplus Food Rescue & NGO Dispatch Platform

Introduction

Food waste is one of the most persistent inefficiencies in the food service industry: restaurants, cafés, and grocery stores routinely prepare or stock more than they sell, and the surplus that remains at closing time is discarded rather than redirected to people who need it. At the same time, local shelters, food banks, and community kitchens operate on tight budgets and limited donated supply, often unaware that usable surplus exists just a few streets away. FoodBridge closes this gap in partnership with an NGO that operates its own driver fleet: donors post surplus — by typing a quick description or speaking it aloud — and the system automatically extracts the details, matches it to the nearest receiving organization with capacity, and dispatches an NGO driver to complete the pickup and delivery before the food spoils. The NGO gets a deployable coordination tool and a concrete, reportable impact story; donors and receiving organizations get a system that does the coordination work neither side has time for.

Problem Statement

Restaurants, cafés, and grocery stores routinely discard unsold food at closing time — not because it is unwanted, but because coordinating pickup with a receiving organization is logistically difficult, and even when an NGO is willing to run the logistics, matching donations to the right receiver and dispatching a driver efficiently is still typically done by phone and memory. Receiving organizations have no visibility into what surplus exists nearby at any given moment, so viable donations are missed entirely, while NGO coordinators dispatching drivers by hand have no way to know whether a given assignment is actually the most efficient one available. FoodBridge addresses both problems at once: donors describe surplus in their own words rather than filling out a form, the system matches each listing to the nearest receiver with capacity and prioritizes by urgency, and driver assignments are generated by an optimizer rather than guesswork.

Objectives

- Enable donors to report surplus quickly, using free-text or voice descriptions rather than structured forms.
- Automatically match surplus listings to the nearest receiving organization with sufficient capacity, prioritized by expiration urgency.
- Dispatch the NGO's own drivers to pickups and deliveries in an order that minimizes total travel time, rather than assigning them manually.
- Give receiving organizations visibility into incoming donations and NGO coordinators a live view of dispatch status.
- Provide the NGO with impact reporting it can use with its own donors and funders.

Scope

FoodBridge is built for a single NGO that operates its own driver fleet — donors and receiving organizations remain separate, independent parties, but pickup and delivery logistics run entirely through the NGO rather than an open volunteer marketplace. The system covers four roles: Donors listing surplus, Receivers (shelters, food banks, community kitchens) with defined capacity, Drivers who are part of the NGO's roster rather than public sign-ups, and an NGO Coordinator who oversees matching, dispatch, and verification. Two AI-driven components sit in the core build rather than being deferred: a language-model-based parser that turns a donor's free-text description into a structured listing, and an optimization engine — not a trained model, but a constraint solver — that assigns drivers to jobs and orders their stops. Both are included because they solve the two biggest sources of manual effort in the pipeline: donor data entry and coordinator dispatch decisions. Capabilities such as predictive demand forecasting, automated impact-report drafting, and live GPS driver tracking are treated as Future Scope,

since they add real value but are not necessary to prove out the core rescue-and-dispatch loop within a solo, 140-hour build.

Proposed Solution / Methodology

FoodBridge follows a client-server architecture with a Django REST Framework backend, a PostgreSQL/PostGIS database, and a React frontend. A donor reports surplus by typing, or speaking a description that is transcribed to text; this is sent to an LLM-based parsing step that extracts item type, estimated quantity, and pickup window into structured fields, which the donor reviews and corrects before submitting. The listing is geocoded once, at the donor's profile level rather than per item, and stored as a PostGIS point. A matching engine then runs a proximity query against Receiver locations within a configurable radius, filters by remaining capacity and dietary or storage constraints, and ranks candidates by a weighted score combining distance and time-to-expiry — this stage is unchanged by the NGO-dispatch model, since Receivers remain plural and independent. Once a Receiver accepts a match, dispatch becomes an assignment problem rather than a marketplace: the NGO Coordinator pushes the job to the driver roster, and an OR-Tools-based optimizer determines which available driver should take the job and in what order, when a driver has multiple open stops, to minimize total travel distance. Donors, Receivers, and Drivers are notified by SMS and email at each stage — match proposed, driver assigned, delivery completed — and completed transactions feed the NGO's analytics dashboard.

Features/Modules

- Donor Module – surplus listing creation via free-text or voice input
- AI Intake Parser – LLM-based extraction of item type, quantity, and pickup window from a donor's free-form description
- Receiver Module – organization profiles, capacity limits, and dietary/storage constraints
- Matching Engine – automated proximity, capacity, and urgency-based Donor-to-Receiver matching
- Dispatch Optimization Engine – OR-Tools-based assignment of NGO drivers to jobs, with multi-stop route ordering
- NGO Coordinator Console – dispatch oversight, donor/receiver verification, and dispute resolution
- Notification System – real-time SMS/email alerts across all four roles
- Analytics & Impact Dashboard – impact metrics (meals rescued, food waste diverted) for the NGO's own reporting

Technology Stack

- Frontend: React, Tailwind CSS
- Backend: Django, Django REST Framework (Python)
- Database: PostgreSQL with PostGIS extension
- Geocoding: GeoPy (OpenStreetMap / Nominatim)
- AI Intake Parsing: LLM API (e.g., Claude, OpenAI) for structured extraction from free-text donor input
- Dispatch Optimization: Google OR-Tools (constraint solver for driver-job assignment and route ordering)
- Notifications: Twilio (SMS), Django's built-in email framework (SMTP)
- Authentication: Django's built-in auth system, djangorestframework-simplejwt (role-based: Donor / Receiver / Driver / NGO Coordinator)
- Hosting/Deployment: Vercel (frontend), Render (backend and database)

System Requirements

Hardware: Standard laptop or desktop with at least 8 GB RAM (16 GB recommended).

Software: Python 3.11+, Node.js 18+, PostgreSQL 15+ with the PostGIS extension, Git, an LLM API key for intake parsing, and the ortools Python package.

Browser: Latest two versions of Chrome, Firefox, Edge, or Safari, with JavaScript enabled.

Operating System: Cross-platform — Windows, macOS, or Linux for development; end users need only a modern web browser.

Expected Outcomes

- A working platform where donors report surplus in plain language instead of filling out forms, materially lowering the effort to donate.
- Automated, urgency-ranked matching between donors and receiving organizations, replacing manual phone-based coordination.
- Driver assignments generated by an optimizer rather than guesswork, with a measurably shorter total route than a naïve nearest-driver approach.
- A concrete NGO partnership: the organization gets an operational dispatch tool plus impact metrics it can use with its own donors and funders.
- A solo-built, technically substantial application demonstrating geospatial algorithms, LLM-based data extraction, constraint-based optimization, and role-based access control.

Future Scope

- Predicting Receiver acceptance likelihood from historical match outcomes and folding it into the matching score.
- Driver ETA prediction and anomaly flagging when a driver is running significantly behind schedule.
- A donor reliability score, flagging donors whose listed quantities tend to run smaller than described.
- Demand forecasting to help the NGO anticipate which donors will have surplus on which days.
- Automated drafting of the NGO's donor/funder impact reports from the analytics data.
- Voice-input support layered onto the existing text-based intake parser.
- Live GPS tracking for drivers during active deliveries.
- Expansion to multiple NGO branches or partner organizations, each with its own driver roster.

Conclusion

FoodBridge pairs a real operational partner — an NGO with its own driver fleet — with two targeted AI components that remove the two biggest sources of manual effort in food rescue logistics: donor data entry and dispatch decision-making. The Donor-to-Receiver matching engine, unchanged from the platform's original design, sits alongside a language-model-based intake parser and a constraint-based dispatch optimizer, giving the project a technically substantial core spanning geospatial matching, NLP-based extraction, and combinatorial optimization, while giving the NGO partner a tool that produces a genuine, reportable reduction in food waste.